

BDR - Atividade Aula 12

Aluno: Ronaldo de Avila Ribeiro Junior

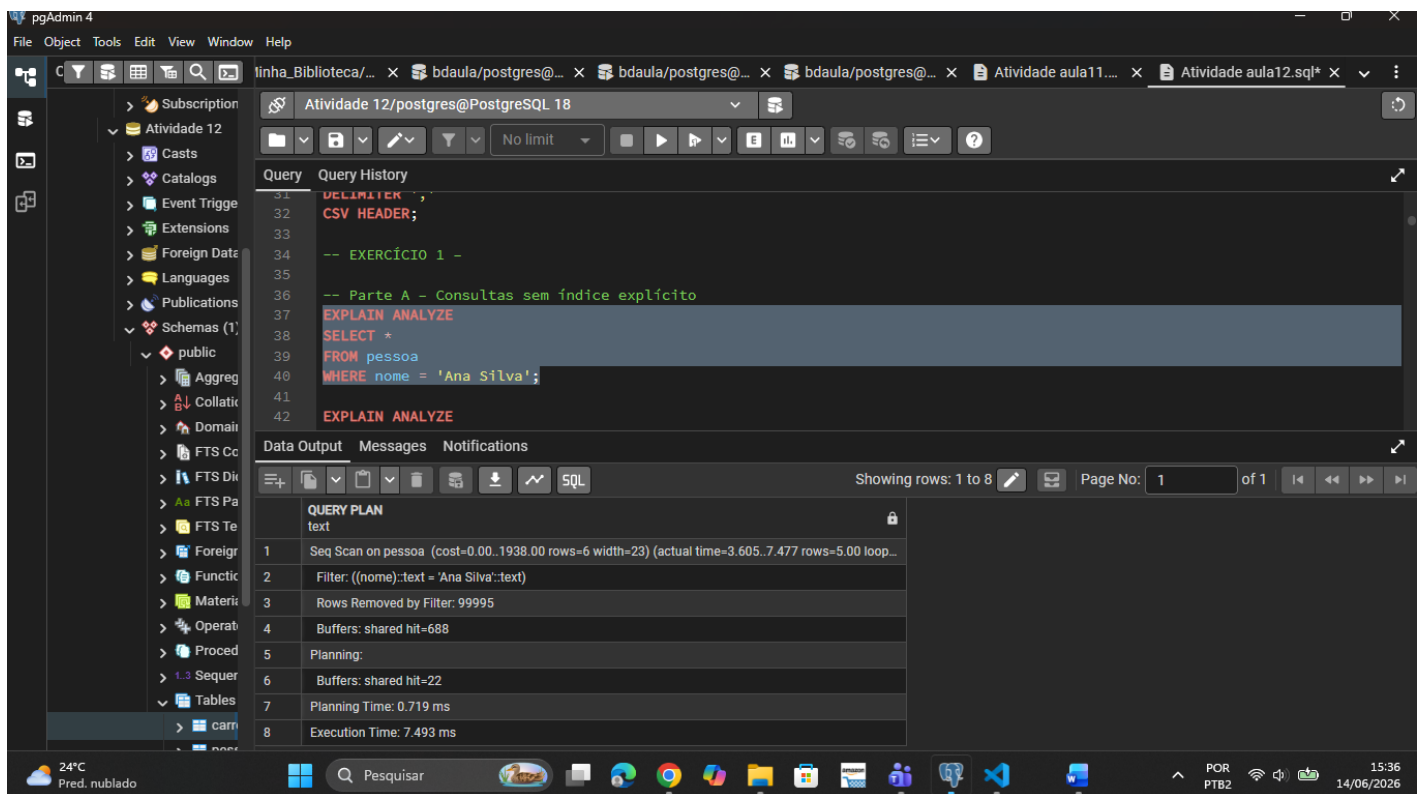
-- *Parte A – Consultas sem índice explícito*

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Ana Silva';



The screenshot shows the pgAdmin 4 interface with a SQL query editor and its execution plan. The query is as follows:

```
DELIMITER ;
CSV HEADER;

-- EXERCÍCIO 1 -
-- Parte A - Consultas sem índice explícito
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Silva';
EXPLAIN ANALYZE
```

The execution plan (QUERY PLAN) is displayed below the query:

Step	Operation	Cost	Actual Time	Rows	Width	Loop Count
1	Seq Scan on pessoa	(cost=0.00..1938.00 rows=6 width=23)	(actual time=3.605..7.477 rows=5.00 loop=1)	5	23	1
2	Filter: ((nome)::text = 'Ana Silva')::text			5		
3	Rows Removed by Filter: 99995					
4	Buffers: shared hit=688					
5	Planning:					
6	Buffers: shared hit=22					
7	Planning Time: 0.719 ms					
8	Execution Time: 7.493 ms					

The interface also shows the pgAdmin 4 menu bar, a sidebar with database objects, and a status bar at the bottom indicating the system temperature (24°C) and date (14/06/2026).

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'João Santos';

The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL code:

```
37 EXPLAIN ANALYZE
38 SELECT *
39 FROM pessoa
40 WHERE nome = 'Ana Silva';
41
42 EXPLAIN ANALYZE
43 SELECT *
44 FROM pessoa
45 WHERE nome = 'João Santos';
46
47 -- Parte B - Criação do índice B-tree em nome
```

The Data Output tab shows the query plan for the second query:

Step	Operation	Details
1	Seq Scan on pessoa	(cost=0.00..1938.00 rows=6 width=23) (actual time=3.890..8.955 rows=6.00 loop...)
2	Filter: ((nome)::text = 'João Santos'::text)	
3	Rows Removed by Filter: 99994	
4	Buffers: shared hit=688	
5	Planning Time: 0.114 ms	
6	Execution Time: 8.976 ms	

-- Parte B – Criação do índice B-tree em nome

CREATE INDEX idx_pessoa_nome

ON pessoa (nome);

The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL code:

```
42 EXPLAIN ANALYZE
43 SELECT *
44 FROM pessoa
45 WHERE nome = 'João Santos';
46
47 -- Parte B - Criação do índice B-tree em nome
48 CREATE INDEX idx_pessoa_nome
49 ON pessoa (nome);
50
51 -- Parte C - Consultas com índice
52 EXPLAIN ANALYZE
```

The Data Output tab shows the successful execution of the CREATE INDEX statement:

```
CREATE INDEX
Query returned successfully in 537 msec.
```

-- Parte C – Consultas com índice

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Ana Silva';

The screenshot shows the pgAdmin 4 interface with the query execution results for the query: `EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'Ana Silva';`. The query plan is displayed in the 'Data Output' tab, showing the following steps:

Step	Operation	Cost	Actual Time	Actual Rows	Actual Loops
1	Bitmap Heap Scan on pessoa	(cost=4.34..26.73 rows=6 width=23)	(actual time=0.074..0.078 rows=5.00 loops=1)		
2	Recheck Cond: ((nome)::text = 'Ana Silva')::text				
3	Heap Blocks: exact=5				
4	Buffers: shared hit=5 read=2				
5	Bitmap Index Scan on idx_pessoa_nome	(cost=0.00..4.34 rows=6 width=0)	(actual time=0.057..0.057 rows=5.00 loops=1)		
6	Index Cond: ((nome)::text = 'Ana Silva')::text				
7	Index Searches: 1				
8	Buffers: shared read=2				
9	Planning:				
10	Buffers: shared hit=15 read=1				
11	Planning Time: 0.456 ms				

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'João Santos';

The screenshot shows the pgAdmin 4 interface with the query execution results for the query: `EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'João Santos';`. The query plan is displayed in the 'Data Output' tab, showing the following steps:

Step	Operation	Cost	Actual Time	Actual Rows	Actual Loops
1	Bitmap Heap Scan on pessoa	(cost=4.34..26.73 rows=6 width=23)	(actual time=0.122..0.130 rows=6.00 loops=1)		
2	Recheck Cond: ((nome)::text = 'João Santos')::text				
3	Heap Blocks: exact=6				
4	Buffers: shared hit=7 read=1				
5	Bitmap Index Scan on idx_pessoa_nome	(cost=0.00..4.34 rows=6 width=0)	(actual time=0.101..0.101 rows=6.00 loops=1)		
6	Index Cond: ((nome)::text = 'João Santos')::text				
7	Index Searches: 1				
8	Buffers: shared hit=1 read=1				
9	Planning Time: 0.374 ms				
10	Execution Time: 0.176 ms				

-- Parte D – Respostas:

-- • Qual plano foi utilizado antes do índice?

--R: Foi utilizado o Seq Scan.

-- • Qual plano foi utilizado após o índice?

-- R: Foi utilizado o Bitmap Heap Scan.

-- • Houve redução no tempo de execução

-- R: Sim houve uma redução bem expressiva, onde caiu para aproximando 7ms.

-- • O índice foi utilizado em ambos os nomes testados? Justifique com base na saída do plano.

-- R: sim. Como ambos os nomes possuem poucos registros, o otimizador optou pelo índice em ambos os casos.

-- EXERCÍCIO 2 – Seletividade e decisão do otimizador

-- **Parte A** – *Remove índice anterior e executa consulta por intervalo*

```
DROP INDEX IF EXISTS idx_pessoa_nome;
```

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento >= DATE '1970-01-01';
```

The screenshot shows the pgAdmin 4 interface with a SQL query editor and a query plan. The query is as follows:

```
-- EXERCICIO 2 - Seletividade e decisão do otimizador
-- Parte A - Remove índice anterior e executa consulta por intervalo
DROP INDEX IF EXISTS idx_pessoa_nome;

EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '1970-01-01';

-- Parte B - Criação do índice em nascimento
CREATE INDEX idx_pessoa_nascimento
```

The query plan shows the following steps:

Step	Operation	Cost	Time
1	Seq Scan on pessoa	(cost=0.00..1938.00 rows=57177 width=23)	(actual time=0.016..8.734 rows=57035.00)
2	Filter: (nascimento >= '1970-01-01'::date)		
3	Rows Removed by Filter: 42965		
4	Buffers: shared hit=688		
5	Planning:		
6	Buffers: shared hit=4		
7	Planning Time: 0.213 ms		
8	Execution Time: 0.460 ms		

-- **Parte B** – Criação do índice em nascimento

CREATE INDEX idx_pessoa_nascimento

ON pessoa (nascimento);

The screenshot shows the pgAdmin 4 interface with the same SQL query editor. The query is now:

```
-- Parte B - Criação do índice em nascimento
CREATE INDEX idx_pessoa_nascimento
ON pessoa (nascimento);

-- Parte C - Consulta com índice criado
EXPLAIN ANALYZE
SELECT *
```

The query plan shows the following steps:

Step	Operation	Cost	Time
1	Seq Scan on pessoa	(cost=0.00..1938.00 rows=57177 width=23)	(actual time=0.016..8.734 rows=57035.00)
2	Filter: (nascimento >= '1970-01-01'::date)		
3	Rows Removed by Filter: 42965		
4	Buffers: shared hit=688		
5	Planning:		
6	Buffers: shared hit=4		
7	Planning Time: 0.213 ms		
8	Execution Time: 0.460 ms		

The Data Output tab shows the following message:

```
CREATE INDEX
Query returned successfully in 114 msec.
```

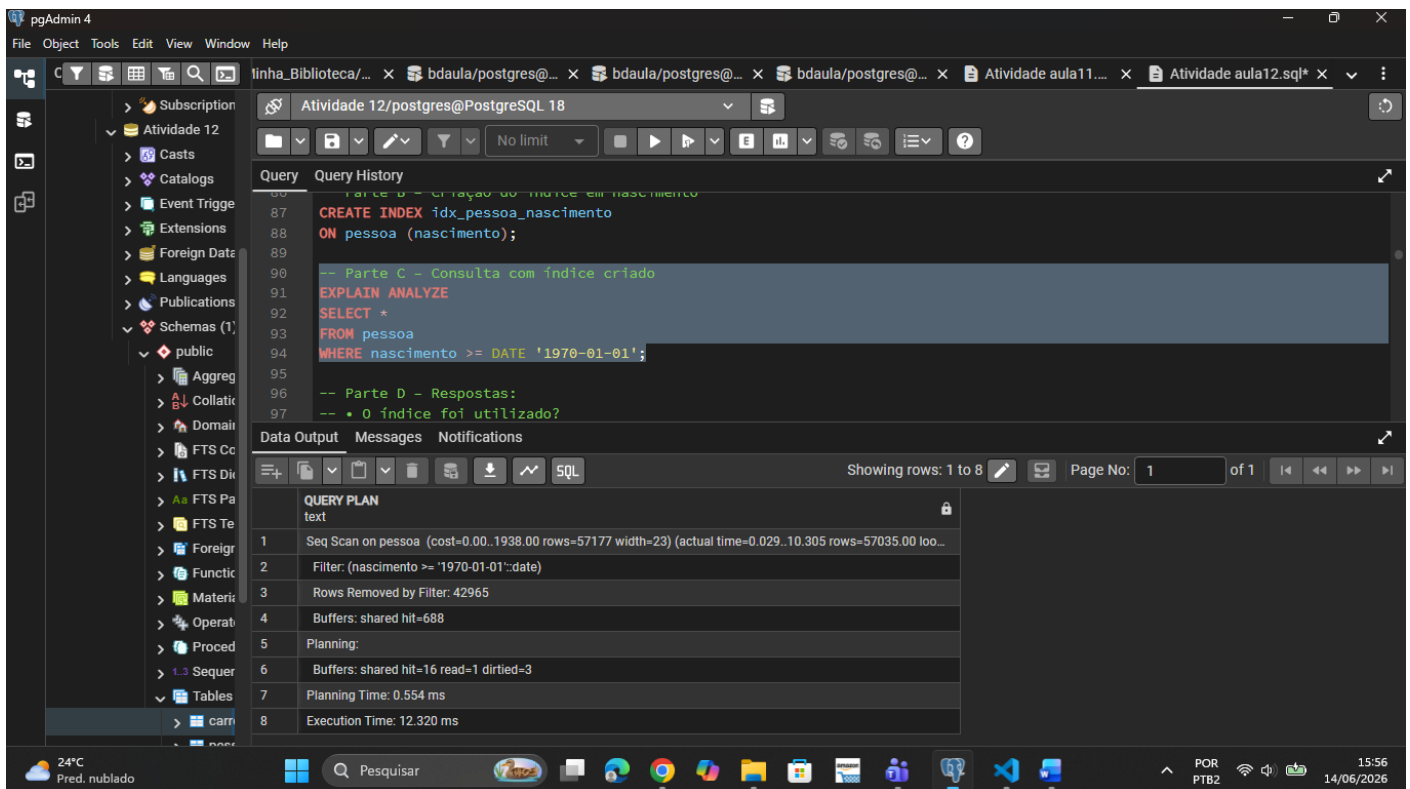
-- **Parte C** – Consulta com índice criado

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '1970-01-01';



-- **Parte D** – Respostas:

-- • O índice criado foi utilizado pelo PostgreSQL?

-- R: Não ele não utilizou

-- • O plano de execução mudou após a criação do índice?

-- R: Não, continuou sendo Seq Scan

-- • Houve melhora significativa no tempo de execução?

-- R: Não, apenas uma mínima melhora na verdade.

-- • Por que o otimizador pode optar por não utilizar um índice, mesmo quando ele existe?

-- R: Quando a consulta retorna uma grande proporção de linhas, o custo de acessar o índice e depois buscar cada bloco individualmente na heap é maior do que simplesmente varrer a tabela de forma sequencial. Nesse caso, o Seq Scan é mais eficiente. O otimizador decide com base em estatísticas de custo estimado.

-- EXERCÍCIO 3 – Índice Composto e Ordem das Colunas

-- **Parte A** – Remove índice anterior e consulta sem índice composto

DROP INDEX IF EXISTS idx_pessoa_nascimento;

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '2000-01-01'

AND nome = 'Ana Silva';

The screenshot shows the pgAdmin 4 interface with a SQL query editor and a query plan window. The query being executed is:

```
-- Parte A - Remove índice anterior e consulta sem índice composto
DROP INDEX IF EXISTS idx_pessoa_nascimento;

EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '2000-01-01'
AND nome = 'Ana Silva';
```

The query plan window displays the following information:

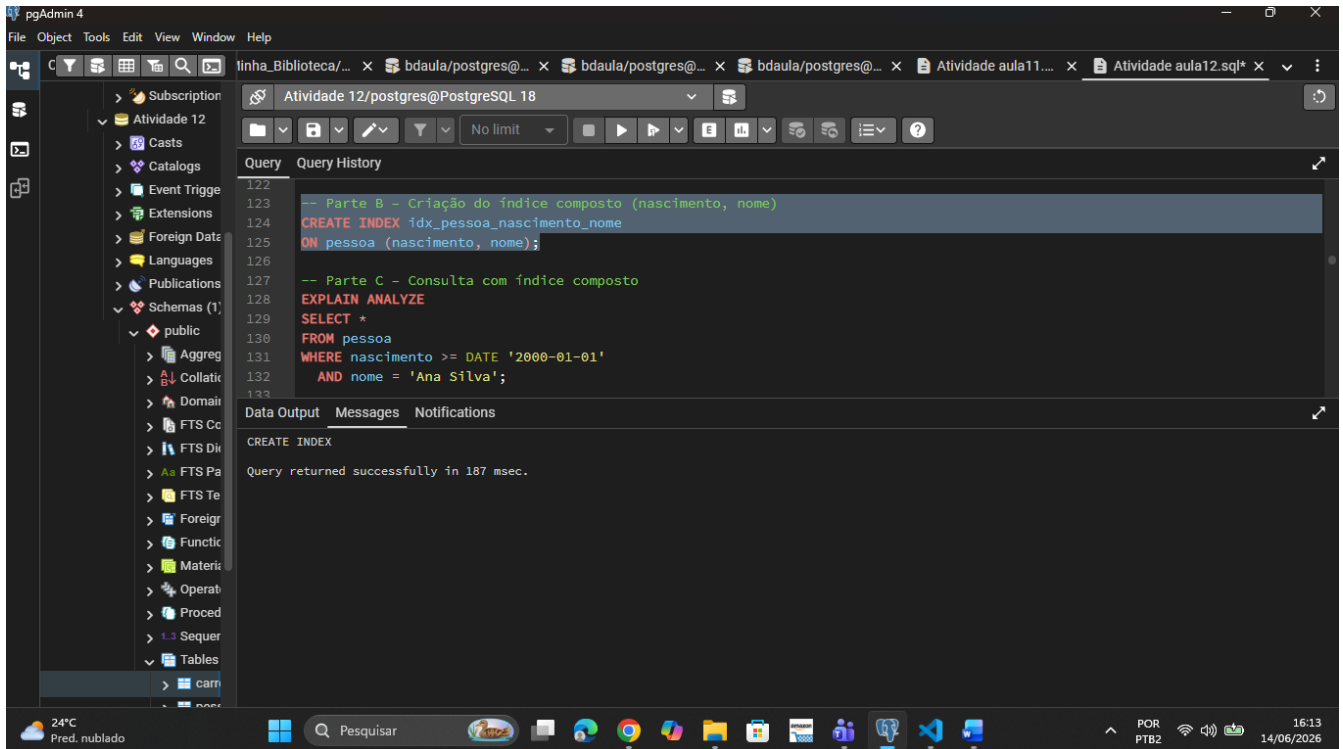
QUERY PLAN
Seq Scan on pessoa (cost=0.00..2188.00 rows=1 width=23) (actual time=5.724..7.963 rows=2.00 loop...)
Filter: ((nascimento >= '2000-01-01'::date) AND ((nome)::text = 'Ana Silva'::text))
Rows Removed by Filter: 99998
Buffers: shared hit=688
Planning:
Buffers: shared hit=4
Planning Time: 0.268 ms
Execution Time: 7.984 ms

The interface also shows the database structure on the left, including schemas like 'public' and 'pg_catalog'.

-- **Parte B** – Criação do índice composto (nascimento, nome)

CREATE INDEX idx_pessoa_nascimento_nome

ON pessoa (nascimento, nome);



-- **Parte C** – Consulta com índice composto

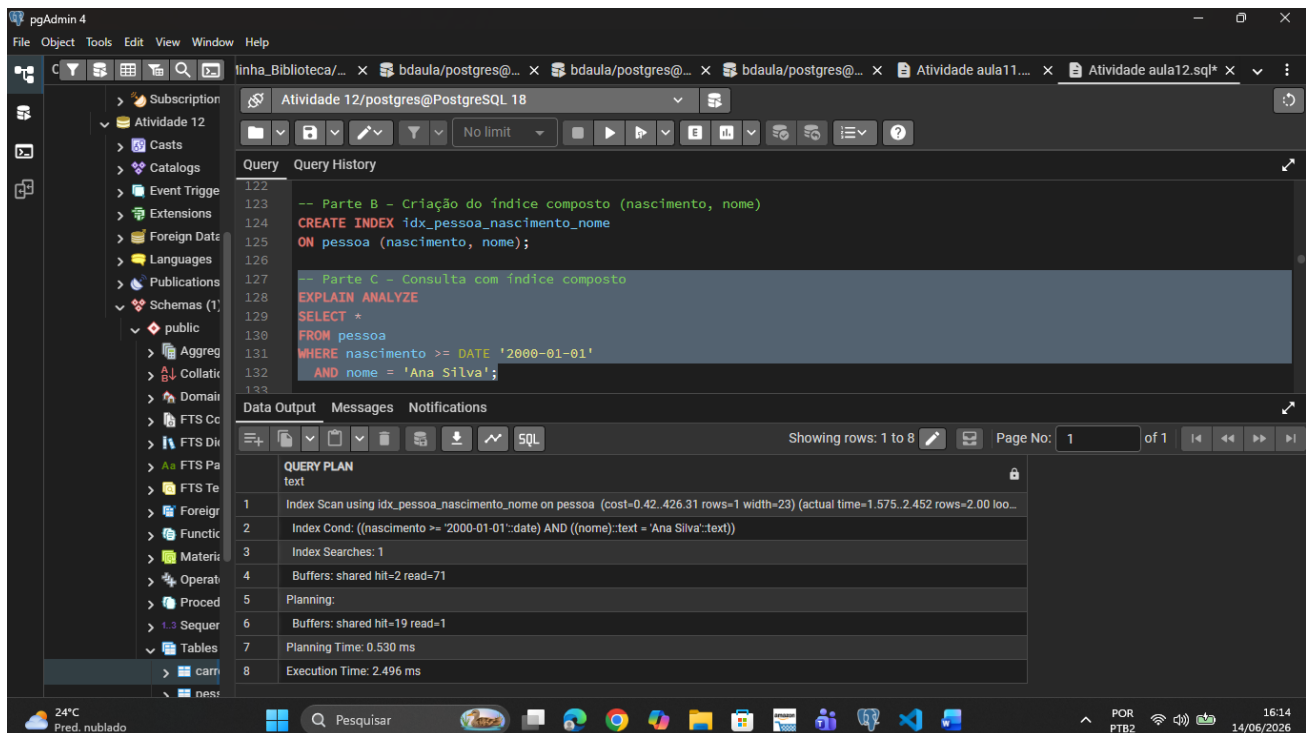
EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '2000-01-01'

AND nome = 'Ana Silva';



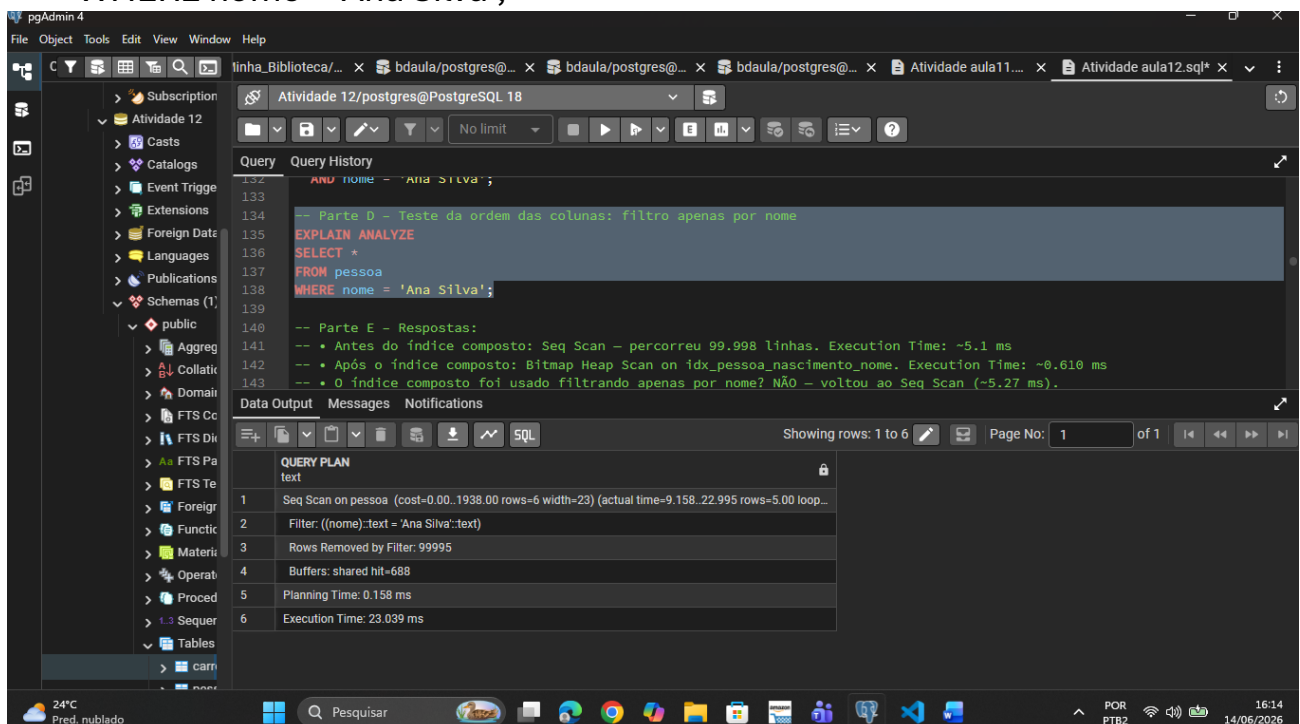
-- Parte D – Teste da ordem das colunas: filtro apenas por nome

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Ana Silva';



-- **Parte E** – Respostas:

-- •• Qual plano foi utilizado antes da criação do índice composto?

-- R: Foi utilizado o Seq Scan.

-- • Qual plano foi utilizado após a criação do índice composto?

--R: Foi utilizado o Index scan.

-- •• O índice composto foi utilizado na consulta que filtra apenas por nome?

--R: NÃO foi.

-- • Por que a ordem das colunas no índice composto é relevante?

--R: Em um índice composto B-tree, os dados são ordenados primeiramente pela 1ª coluna, depois pela 2ª. O índice só pode ser percorrido sequencialmente a partir do prefixo inicial. Onde sem a coluna da esquerda na condição, não há como navegar eficientemente na estrutura, o índice não é aproveitado como deve ser.

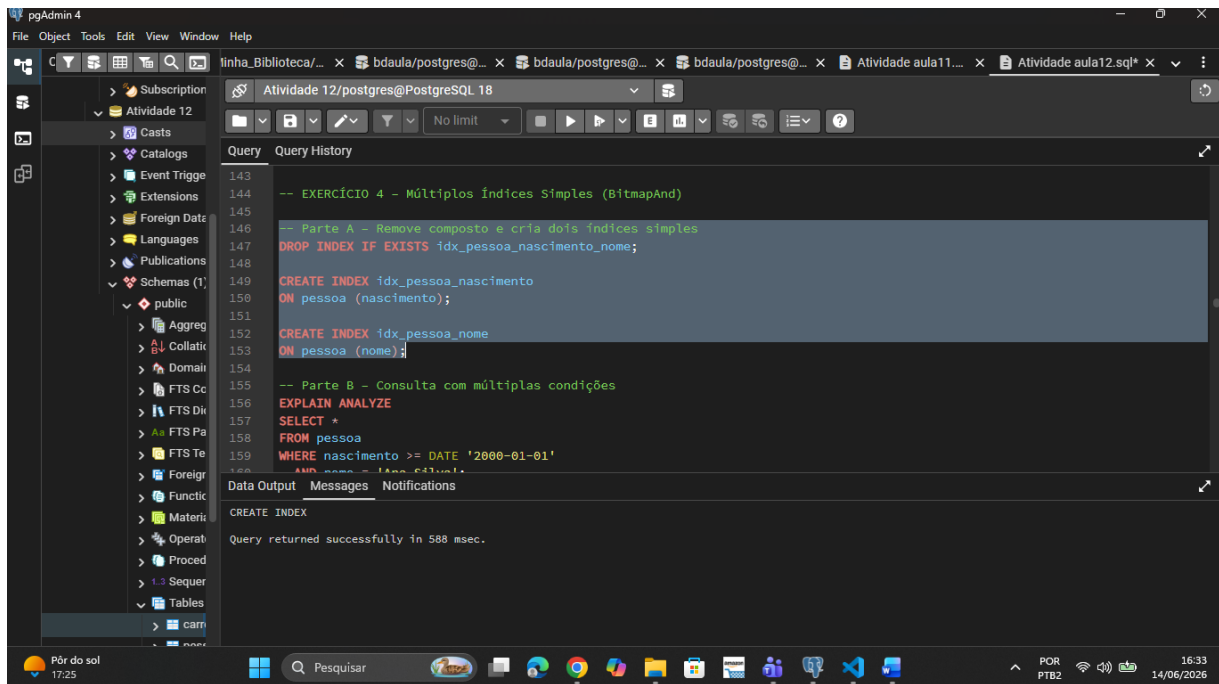
-- **EXERCÍCIO 4 – Múltiplos Índices Simples (BitmapAnd)**

-- **Parte A** – Remove composto e cria dois índices simples

```
DROP INDEX IF EXISTS idx_pessoa_nascimento_nome;
```

```
CREATE INDEX idx_pessoa_nascimento  
ON pessoa (nascimento);
```

```
CREATE INDEX idx_pessoa_nome  
ON pessoa (nome);
```



-- **Parte B** – Consulta com múltiplas condições

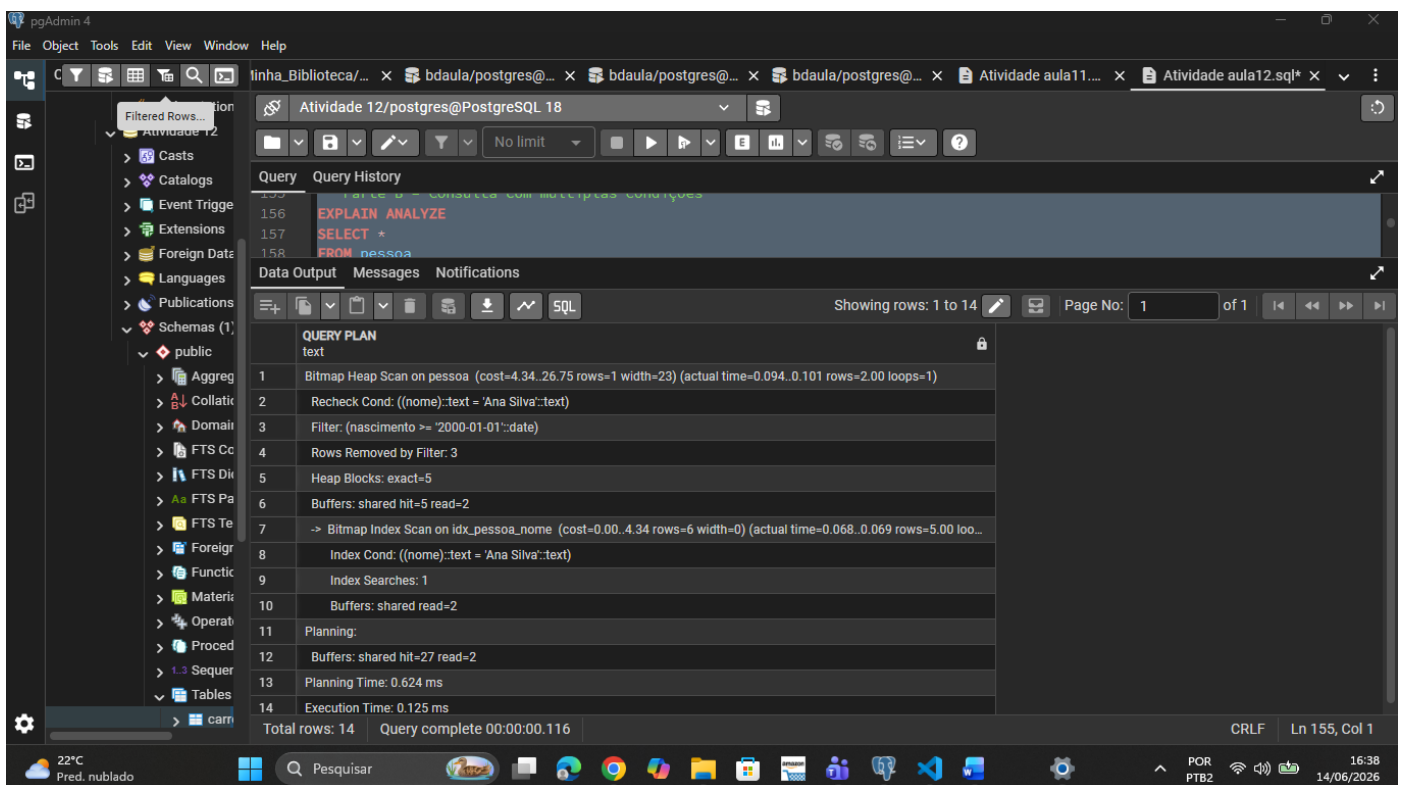
EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '2000-01-01'

AND nome = 'Ana Silva';



-- **Parte C** – Respostas:

--• O PostgreSQL utilizou os dois índices simples na consulta?

--R: Sim, utilizou os dois índices.

--• Qual é o papel do operador BitmapAnd no plano de execução?

--R: Foi o de combinar múltiplos index.

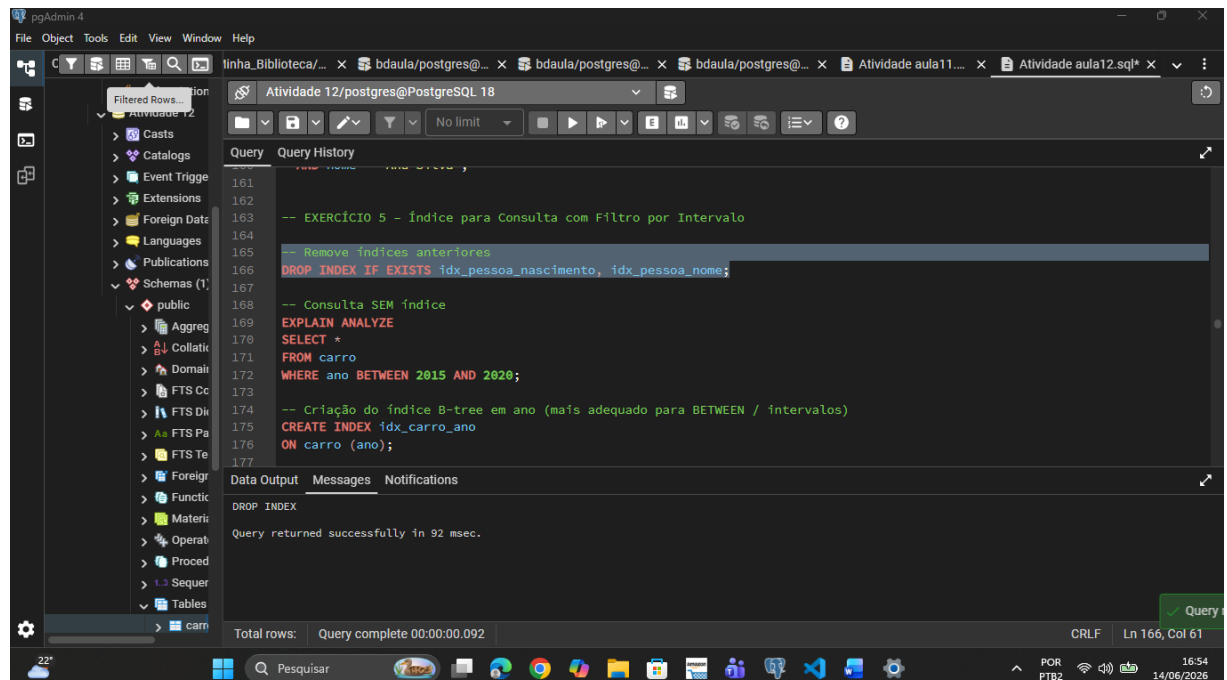
--• O que o plano efetivamente fez?

--R: Ele acabou realizando a leitura dos dois índices separadamente e combinou os resultados.

-- EXERCÍCIO 5 – Índice para Consulta com Filtro por Intervalo

-- Remove índices anteriores

DROP INDEX IF EXISTS idx_pessoa_nascimento, idx_pessoa_nome;



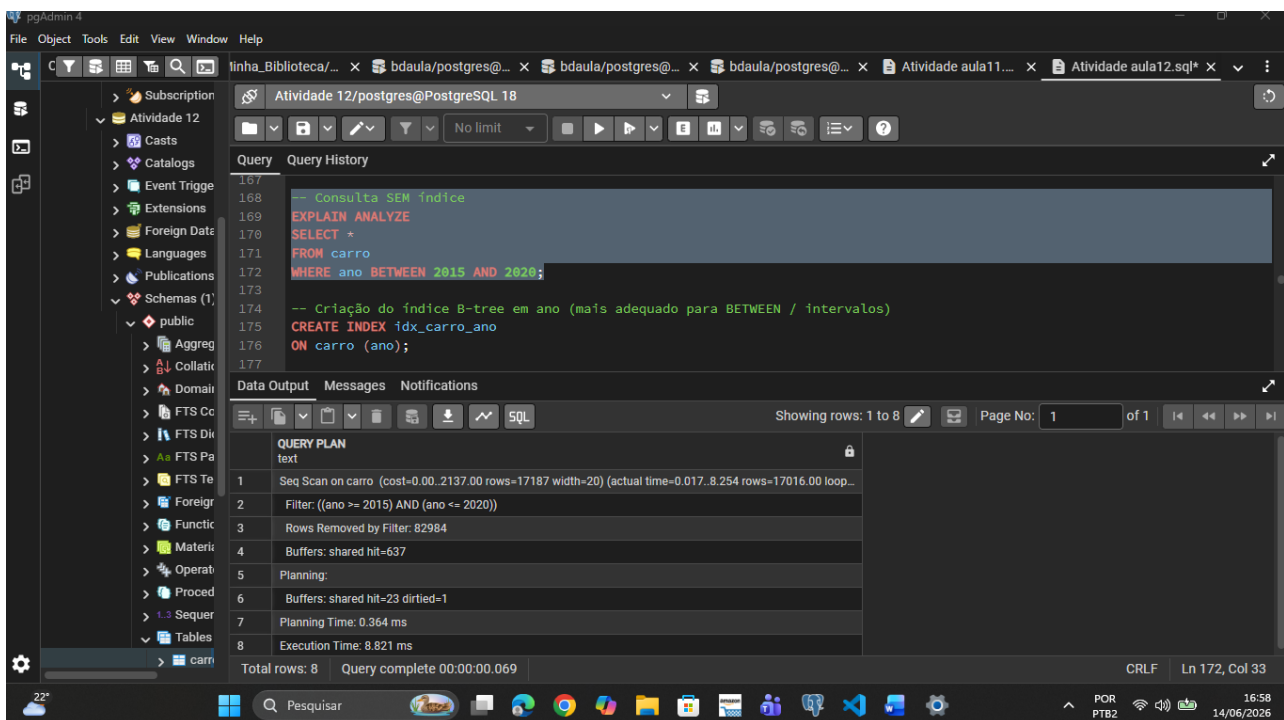
-- Consulta **sem** índice

EXPLAIN ANALYZE

SELECT *

FROM carro

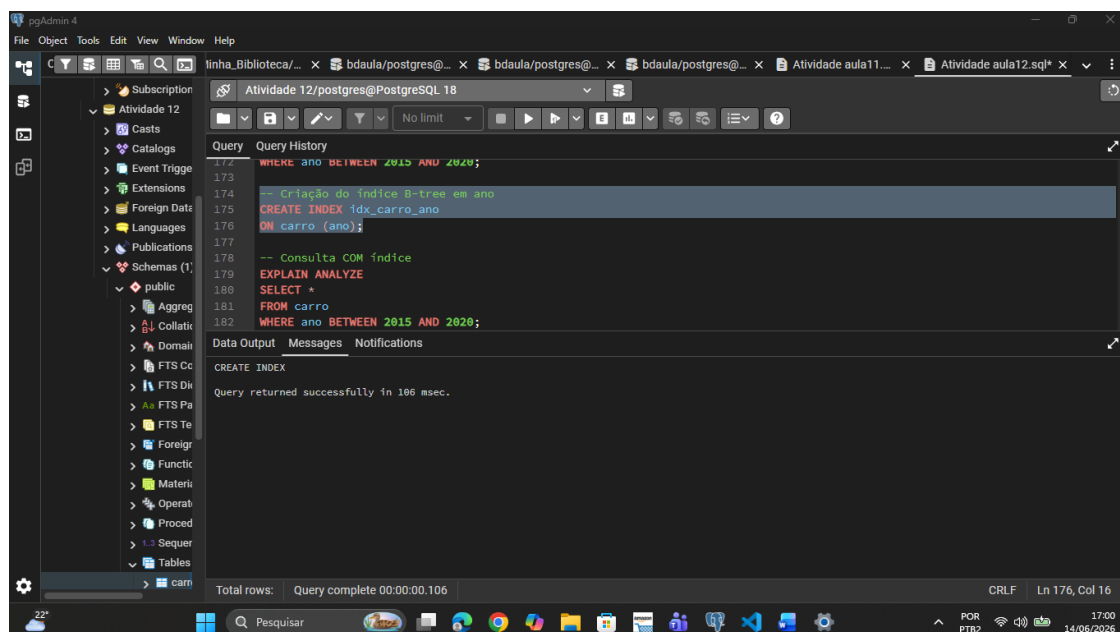
WHERE ano BETWEEN 2015 AND 2020;



-- Criação do índice B-tree em ano

CREATE INDEX idx_carro_ano

ON carro (ano);



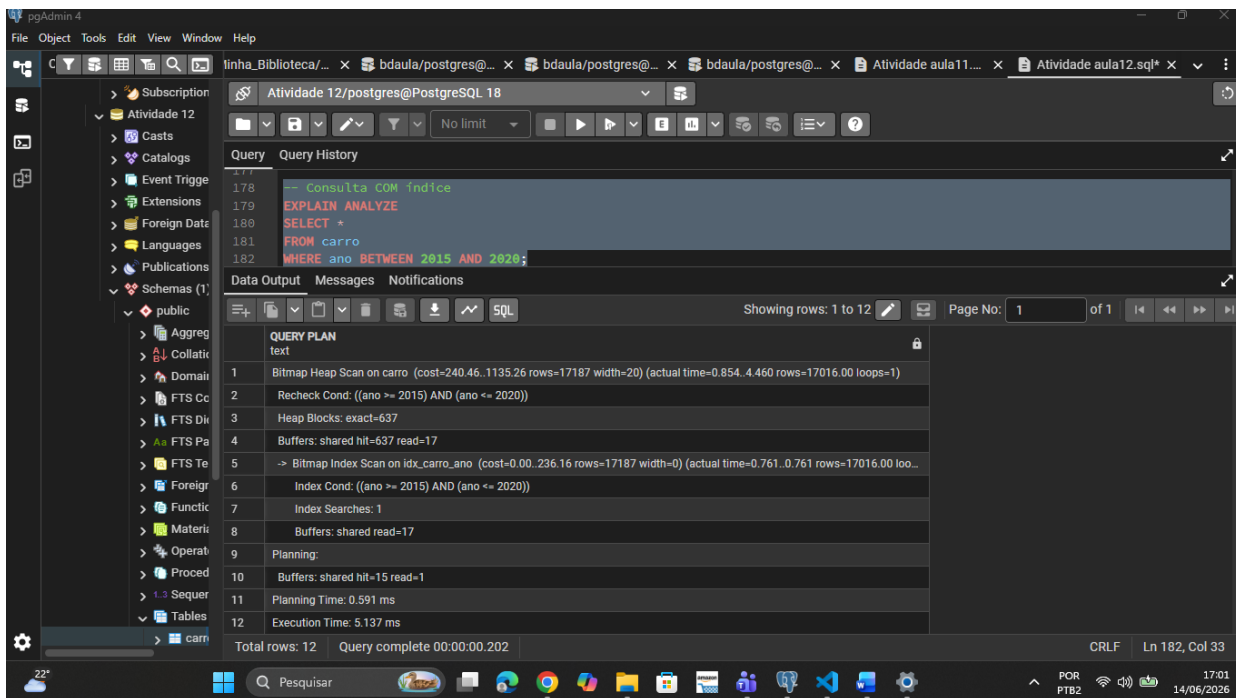
-- Consulta com índice

EXPLAIN ANALYZE

SELECT *

FROM carro

WHERE ano BETWEEN 2015 AND 2020;



-- **Comparação:**

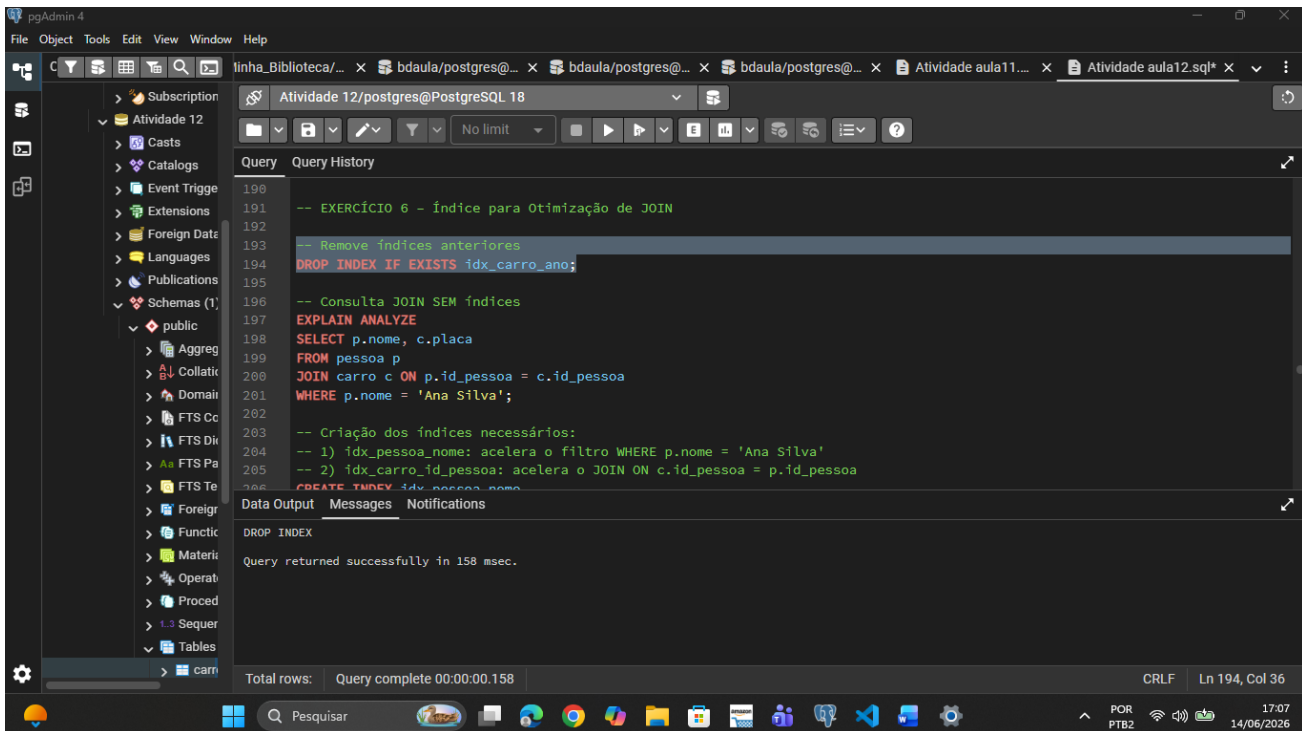
-- **Sem índice:** Seq Scan — 82.984 linhas removidas. Tempo de execução: 7.67 ms

-- **Com índice:** Bitmap Heap Scan on idx_carro_ano. Tempo de execução: 3.74 ms. Uma ótima melhora, pois, o índice B-tree é ideal para BETWEEN, pois, os dados ficam ordenados, permitindo localizar o intervalo diretamente na árvore.

-- **EXERCÍCIO 6 – Índice para Otimização de JOIN**

-- Remove índices anteriores

DROP INDEX IF EXISTS idx_carro_ano;



-- Consulta JOIN SEM índices

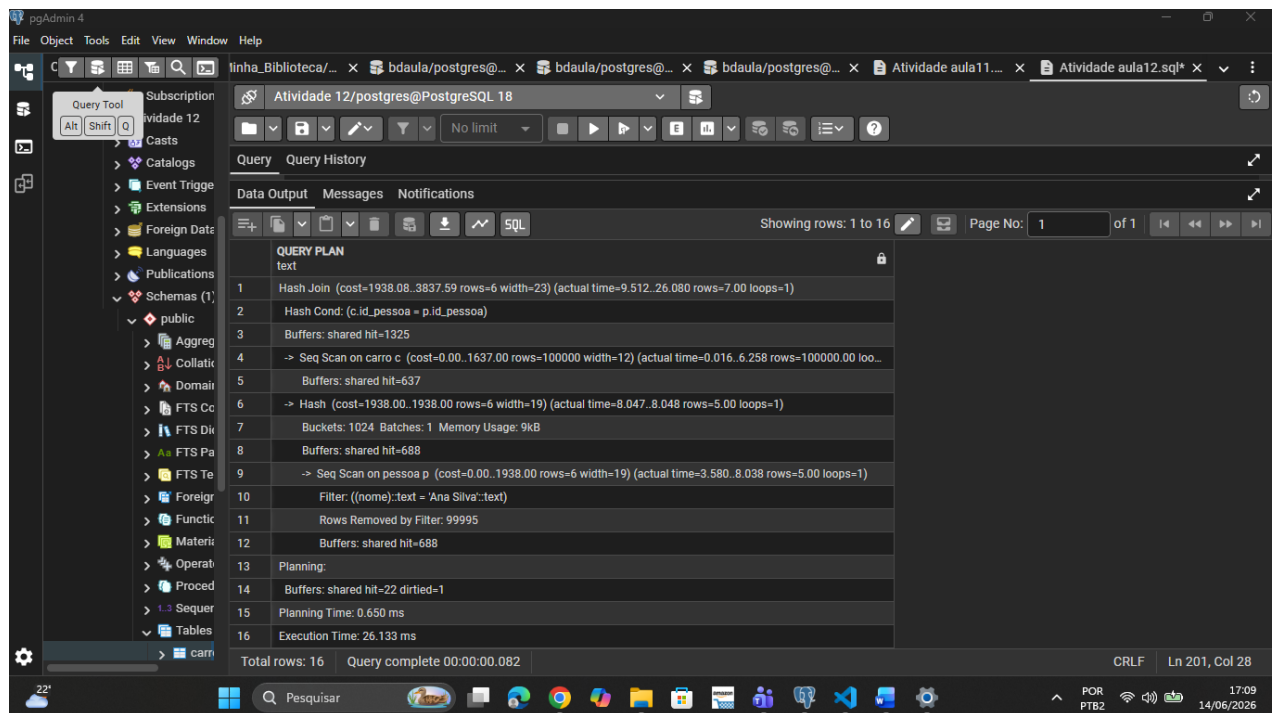
EXPLAIN ANALYZE

SELECT p.nome, c.placa

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nome = 'Ana Silva';



-- Criação dos índices necessários:

-- 1) idx_pessoa_nome: acelera o filtro WHERE p.nome = 'Ana Silva'

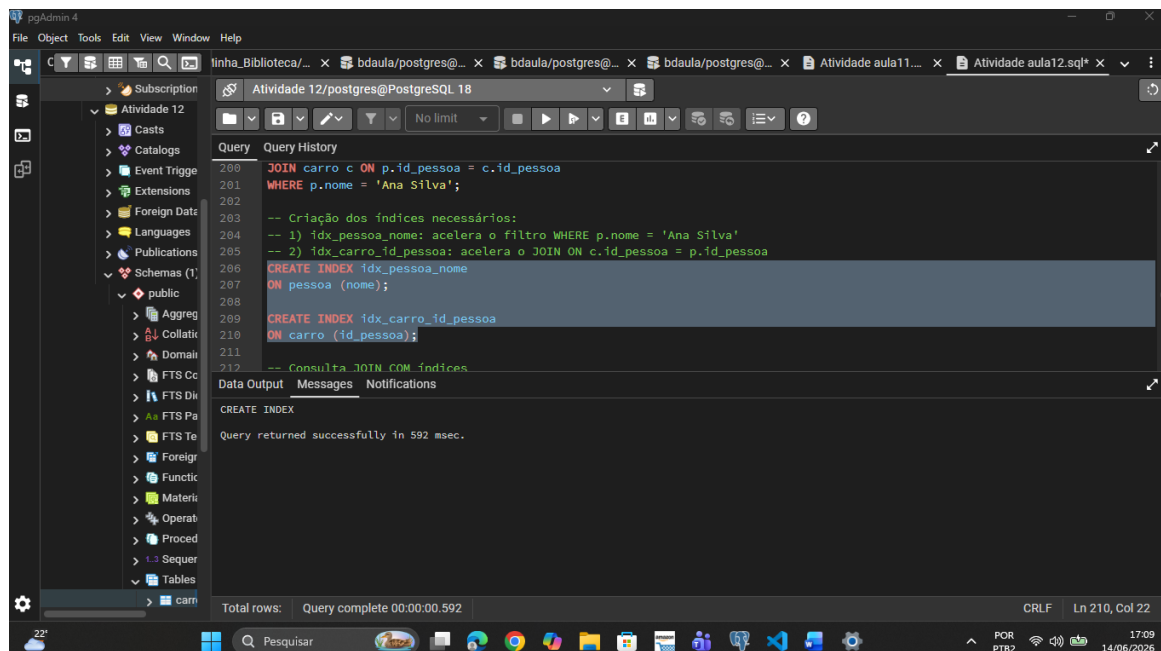
-- 2) idx_carro_id_pessoa: acelera o JOIN ON c.id_pessoa = p.id_pessoa

CREATE INDEX idx_pessoa_nome

ON pessoa (nome);

CREATE INDEX idx_carro_id_pessoa

ON carro (id_pessoa);



-- Consulta JOIN COM índices

EXPLAIN ANALYZE

SELECT p.nome, c.placa

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nome = 'Ana Silva';

The screenshot shows the pgAdmin 4 interface with a query window titled 'Atividade 12/postgres@PostgreSQL 18'. The query executed is: `EXPLAIN ANALYZE SELECT p.nome, c.placa FROM pessoa p JOIN carro c ON p.id_pessoa = c.id_pessoa WHERE p.nome = 'Ana Silva';`. The query plan is displayed in the 'Data Output' tab, showing the following details:

- 1. Nested Loop (cost=8.65..98.83 rows=6 width=23) (actual time=0.069..0.132 rows=7.00 loops=1)
- 2. Buffers: shared hit=24
- 3. -> Bitmap Heap Scan on pessoa p (cost=4.34..26.73 rows=6 width=19) (actual time=0.051..0.061 rows=5.00 loops=1)
- 4. Recheck Cond: ((nome)::text = 'Ana Silva':text)
- 5. Heap Blocks: exact=5
- 6. Buffers: shared hit=7
- 7. -> Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.027..0.028 rows=5.00 loops=1)
- 8. Index Cond: ((nome)::text = 'Ana Silva':text)
- 9. Index Searches: 1
- 10. Buffers: shared hit=2
- 11. -> Bitmap Heap Scan on carro c (cost=4.31..12.00 rows=2 width=12) (actual time=0.010..0.010 rows=1.40 loops=5)
- 12. Recheck Cond: (p.id_pessoa = id_pessoa)
- 13. Heap Blocks: exact=7
- 14. Buffers: shared hit=17
- 15. -> Bitmap Index Scan on idx_carro_id_pessoa (cost=0.00..4.31 rows=2 width=0) (actual time=0.005..0.005 rows=1.40 loops=5)
- 16. Index Cond: (id_pessoa = p.id_pessoa)
- 17. Index Searches: 5
- 18. Buffers: shared hit=10
- 19. Planning:
- 20. Buffers: shared hit=12
- 21. Planning Time: 0.443 ms
- 22. Execution Time: 0.176 ms

Total rows: 22 | Query complete 00:00:00.088 | CRLF | Ln 217, Col 28

--Comparação

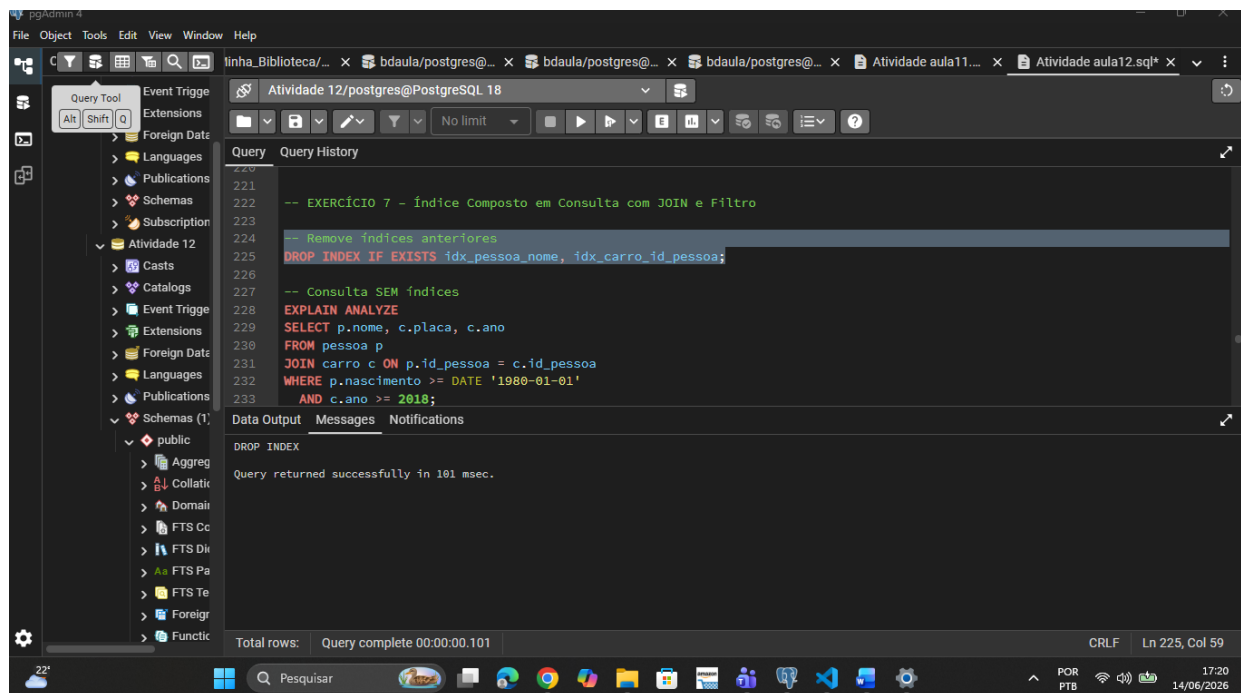
--**Sem índice:** (mais lento) Seq Scan.

--**Com index:** (mais rápido e join mais eficiente) Bitmap Index Scan.

-- EXERCÍCIO 7 – Índice Composto em Consulta com JOIN e Filtro

-- Remove índices anteriores

DROP INDEX IF EXISTS idx_pessoa_nome, idx_carro_id_pessoa;



-- Consulta SEM índices

EXPLAIN ANALYZE

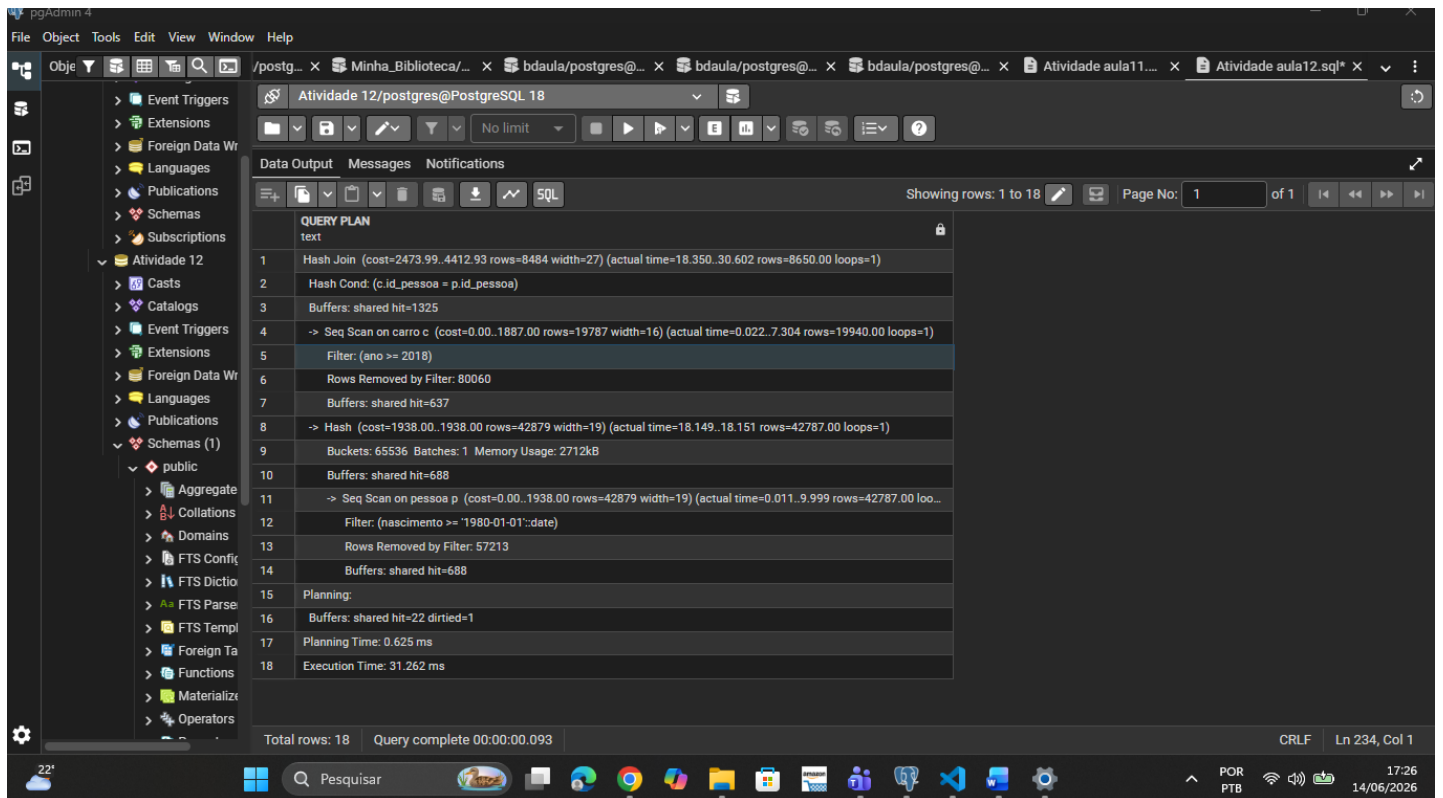
SELECT p.nome, c.placa, c.ano

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nascimento >= DATE '1980-01-01'

AND c.ano >= 2018;



-- Índices criados:

-- pessoa: índice simples em nascimento — filtro de intervalo, coluna única

-- carro: índice composto (ano, id_pessoa) — cobre o filtro em ano E o JOIN em id_pessoa

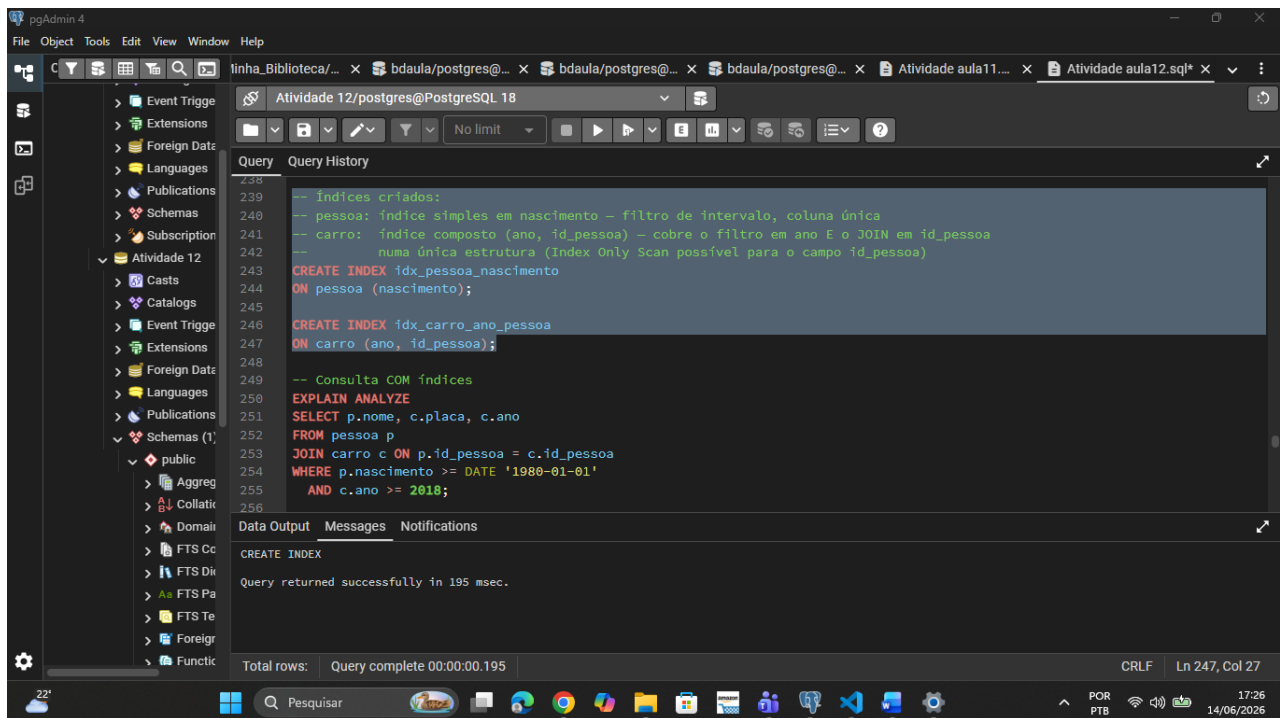
-- numa única estrutura (Index Only Scan possível para o campo id_pessoa)

CREATE INDEX idx_pessoa_nascimento

ON pessoa (nascimento);

CREATE INDEX idx_carro_ano_pessoa

ON carro (ano, id_pessoa);



-- Consulta COM índices

EXPLAIN ANALYZE

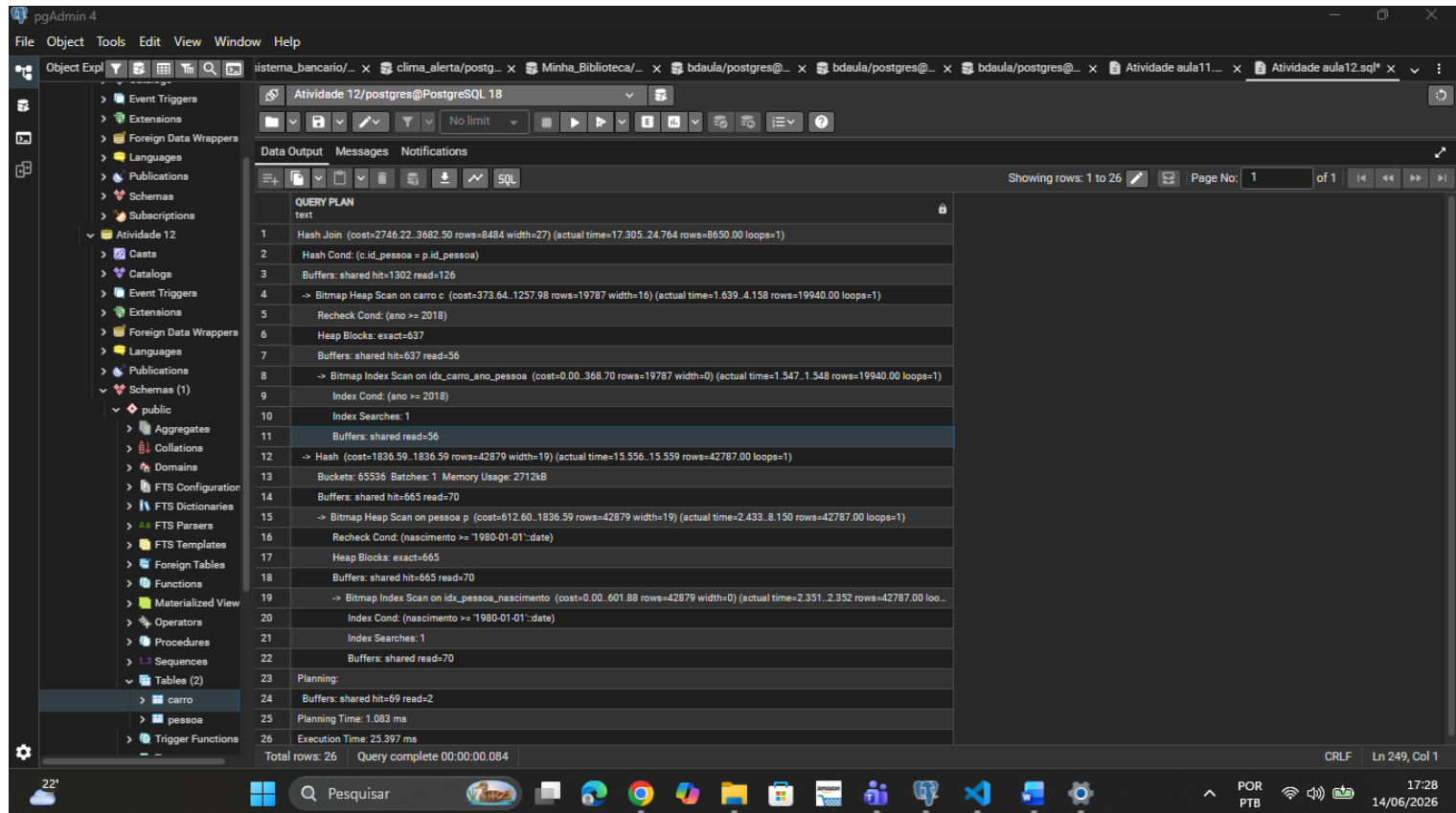
SELECT p.nome, c.placa, c.ano

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nascimento >= DATE '1980-01-01'

AND c.ano >= 2018;



-- Comparação:

-- **Sem índices:** Hash Join + dois Seq Scans. Tempo de execução: 28ms

-- **Com índices:** Hash Join + Bitmap Heap Scan em ambas as tabelas. Tempo de execução: 25 ms

-- **Justificativa dos índices:**

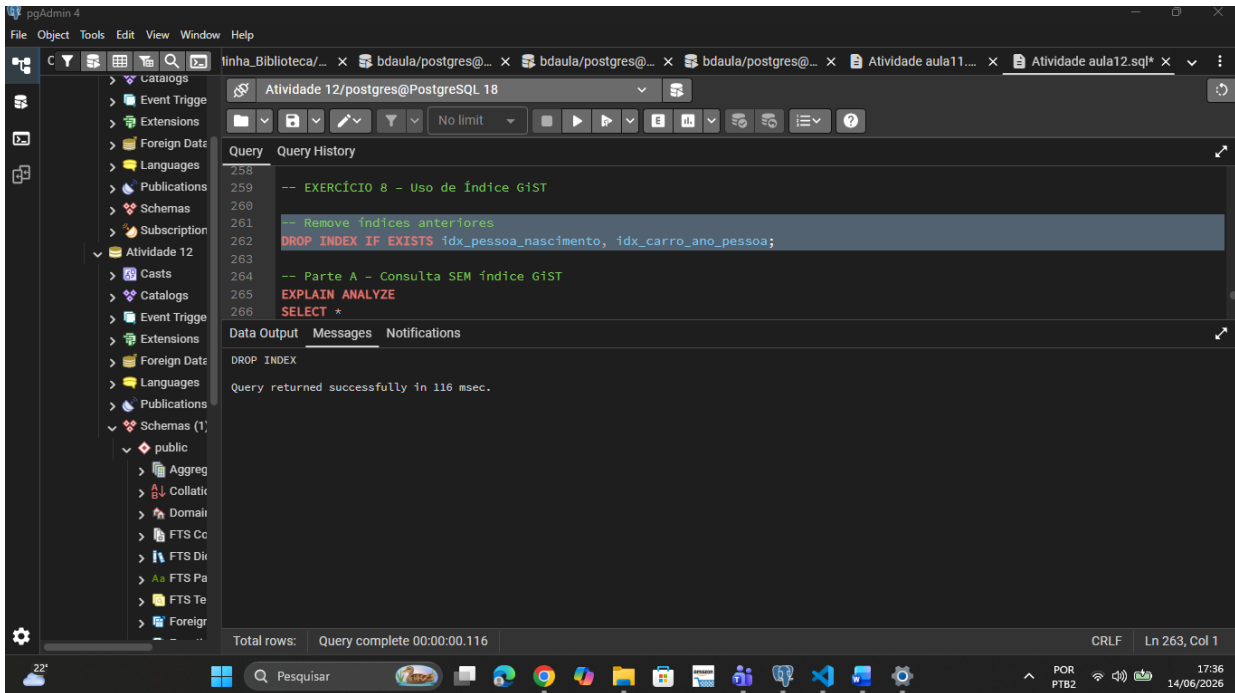
-- • Índice simples em pessoa(nascimento): a coluna é usada isoladamente como filtro de intervalo; não há outra coluna no WHERE para justificar índice composto.

-- • Índice composto carro (ano, id_pessoa): ano é o primeiro filtro (maior poder de redução do intervalo); id_pessoa está incluso para cobrir o JOIN sem acesso adicional à heap (potencial Index Only Scan). Ordem: ano primeiro pois é a coluna com condição de intervalo que restringe mais os registros.

-- EXERCÍCIO 8 – Uso de Índice GiST

-- Remove índices anteriores

```
DROP INDEX IF EXISTS idx_pessoa_nascimento,  
idx_carro_ano_pessoa;
```



-- Parte A – Consulta SEM índice GiST

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
```

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including Schemas (1), public, and various tables. The main window shows a SQL query in the 'Query' tab, which is being executed. The query is as follows:

```
-- Parte A - Consulta SEM índice GiST
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';

-- Parte B - Ativar extensão btree_gist e criar índice GiST
```

The 'Query History' tab shows the execution plan for the query. The plan is as follows:

Step	Operation	Cost	Time	Rows	Width	Bytes
1	Seq Scan on pessoa	(cost=0.00..2188.00 rows=15815 width=23)	(actual time=0.024..10.021 rows=15745.00 loops=1)	15745	23	1048800
2	Filter: ((nascimento >= '1980-01-01'::date) AND (nascimento <= '1990-12-31'::date))			15745	23	1048800
3	Rows Removed by Filter: 84255					
4	Buffers: shared hit=688					
5	Planning:					
6	Buffers: shared hit=5 dirtied=2					
7	Planning Time: 0.345 ms					
8	Execution Time: 10.588 ms					

The 'Data Output' tab shows the results of the query, which are 8 rows. The status bar at the bottom indicates 'Total rows: 8' and 'Query complete 00:00:00.079'.

-- Parte B – Ativar extensão btree_gist e criar índice GiST

CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE INDEX idx_pessoa_nascimento_gist

ON pessoa

USING GIST (nascimento);

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including Schemas (1), public, and various tables. The main window shows a SQL query in the 'Query' tab, which is being executed. The query is as follows:

```
-- Parte B - Ativar extensão btree_gist e criar índice GiST
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE INDEX idx_pessoa_nascimento_gist
ON pessoa
USING GIST (nascimento);

-- Verificar criação do índice
```

The 'Query History' tab shows the execution plan for the query. The plan is as follows:

Step	Operation	Cost	Time	Rows	Width	Bytes
1	Seq Scan on pessoa	(cost=0.00..2188.00 rows=15815 width=23)	(actual time=0.024..10.021 rows=15745.00 loops=1)	15745	23	1048800
2	Filter: ((nascimento >= '1980-01-01'::date) AND (nascimento <= '1990-12-31'::date))			15745	23	1048800
3	Rows Removed by Filter: 84255					
4	Buffers: shared hit=688					
5	Planning:					
6	Buffers: shared hit=5 dirtied=2					
7	Planning Time: 0.345 ms					
8	Execution Time: 10.588 ms					

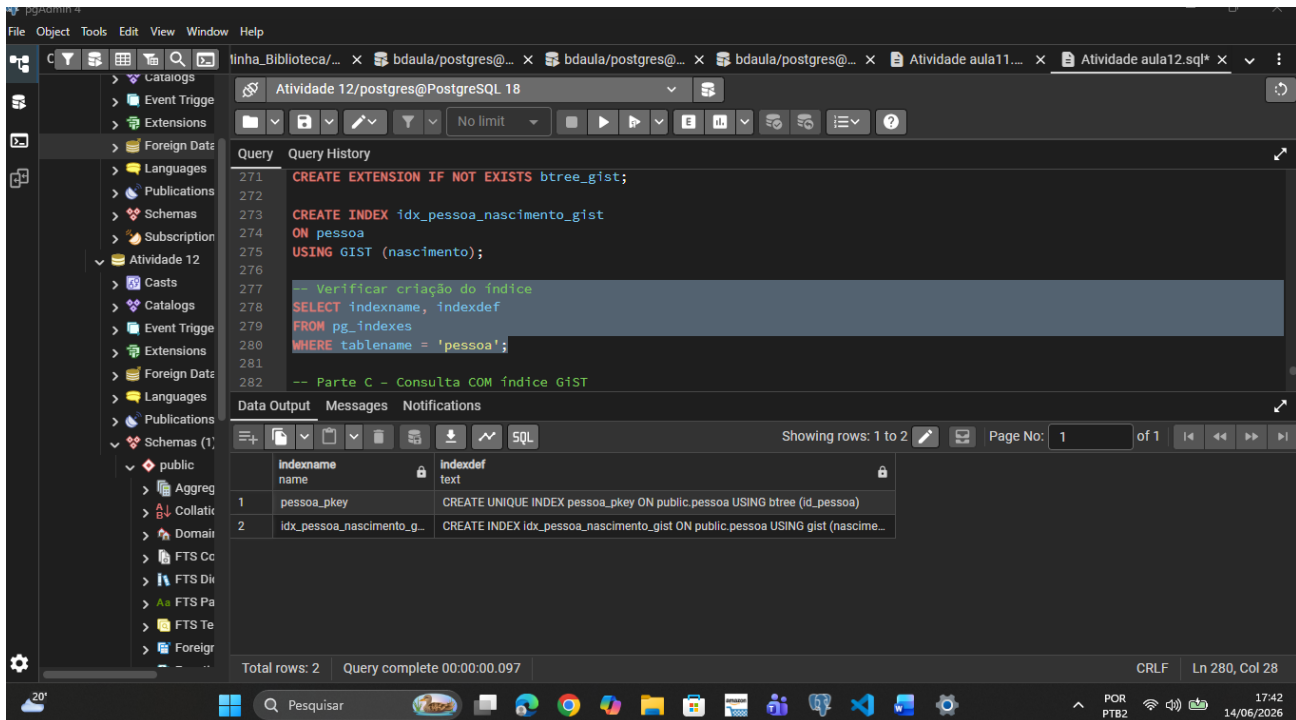
The 'Data Output' tab shows the results of the query, which are 8 rows. The status bar at the bottom indicates 'Total rows: 8' and 'Query complete 00:00:00.084'.

-- Verificar criação do índice

SELECT indexname, indexdef

FROM pg_indexes

WHERE tablename = 'pessoa';



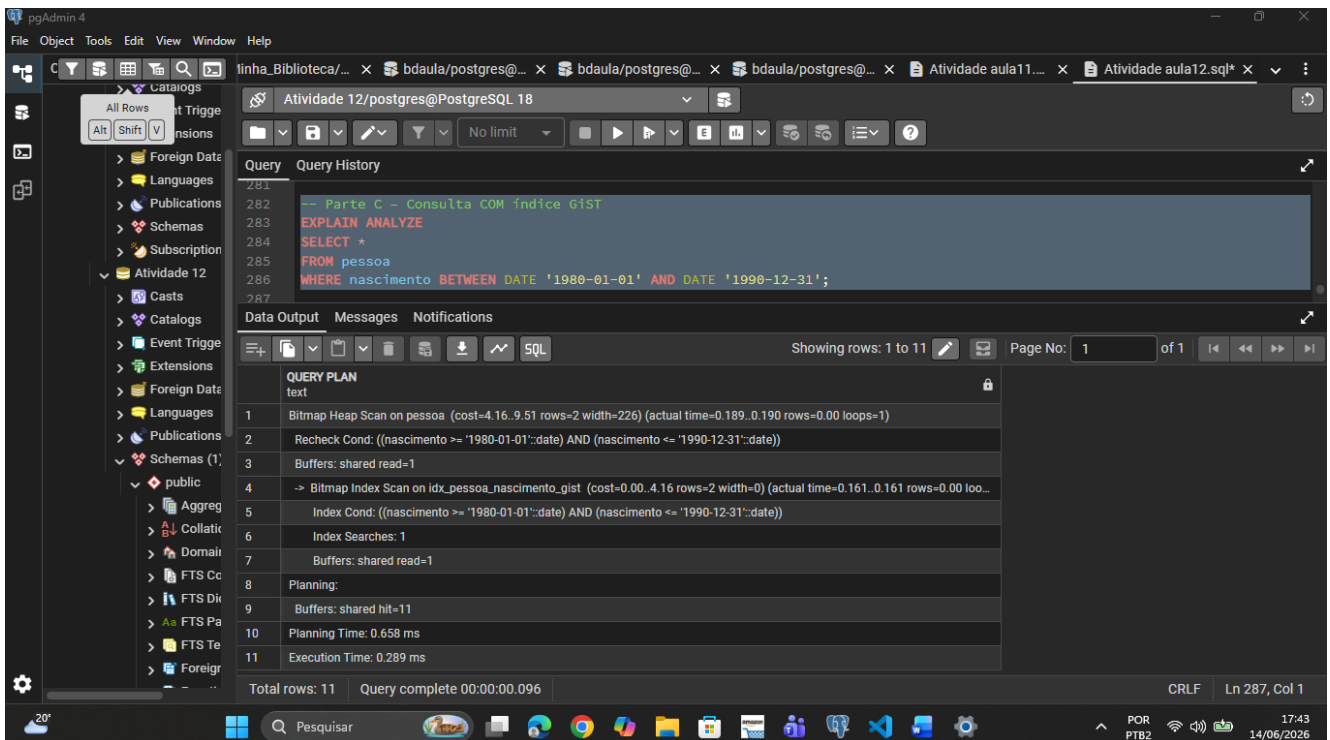
-- Parte C – Consulta COM índice GiST

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';



Parte D – Responda, de forma objetiva

--• Qual plano foi utilizado antes da criação do índice GiST?

--R: Foi utilizado o Seq Scan.

--• O índice GiST foi utilizado após a criação?

--R: Sim ele foi utilizado.

--• Houve melhora no tempo de execução?

--R: Sim houve uma melhoria de aproximadamente de 30% sendo algo de 3 ms. Não foi tanto eficiente quantos os outros utilizados.